



Shaheed Zulfikar Ali Bhutto
Institute of Science & Technology

Smart Savings Planner

Shaheed Zulfikar Ali Bhutto Institute of Science and
Technology

Department of Computer Science

CSC1201: Discrete Mathematical Structures

Project Report

Instructor: Dr. Faria Jameel

By: Naushaba Asif, Rohban Tariq

Registration Id: 2412485, 2412488 [left to right]

Contents

Introduction.....	2
Mathematical Background	2
Discrete Sequences	2
Arithmetic Progression	2
Finite Arithmetic Series	2
Formula Derivation.....	3
Algorithm Description	3
Monthly Savings Plan.....	3
Month Calculation Mode	3
Sample Output / Tables	4
Monthly Savings Plan:.....	4
Month Prediction:	5
Conclusion	6
Java Source Code.....	6

Introduction

Financial planning is a real-world problem that involves making decisions at continuous, fixed intervals. Smart Savings Planner applies concepts from discrete mathematics to model monthly savings behaviour. The amount saved each month is treated as a discrete value, and changes in monthly savings are represented using mathematical sequences. The project uses arithmetic progressions (AP) to calculate total savings, required monthly increases, and the number of months needed to reach a financial target.

The objective of this project is to demonstrate how arithmetic sequences and finite series can be applied to solve practical financial problems in a structured and logical manner.

Mathematical Background

Discrete Sequences

A sequence is an ordered list of numbers defined by a specific rule. Monthly savings naturally form a discrete sequence because each month represents a separate, countable time step. If the amount of money saved changes by a fixed value every month, the sequence of savings can be modelled as an arithmetic sequence.

Arithmetic Progression

An arithmetic progression is sequence in which the difference between consecutive terms is constant. Mathematically Arithmetic sequence is defined by:

$$\text{General Form} = a, a + d, a + 2d, a + 3d, \dots$$

In this project, the first term (a) represents savings in the first month, while the common difference d represents the fixed monthly increase in savings and each term of the sequence represents savings for a specific month.

Finite Arithmetic Series

A finite arithmetic series is the sum of the first n terms of an arithmetic progression. Since savings are calculated over a limited number of months, the total savings form a finite series.

$$\text{Formula: } S_n = \frac{n}{2} [2a + (n - 1)d]$$

Where:

- S_n is the total savings after n months

- a is the first month's saving
- d is the monthly increase in saving
- n is the number of months

This formula created the mathematical foundation of Smart Savings Planner and is used to calculate Savings target and required monthly increase.

Formula Derivation

Savings Planner uses a derived formula from finite arithmetic series:

$$S_n = \frac{n}{2}[2a + (n - 1)d]$$

$$2S_n = n [2a + (n - 1)d]$$

$$\frac{2S_n}{n} = 2a + (n - 1)d$$

$$\frac{2S_n}{n} - 2a = (n - 1)d$$

$$d = \frac{(\frac{2S_n}{n} - 2a)}{(n - 1)}$$

Algorithm Description

Savings Planner is based on two major functional modes: Monthly Savings Plan and month calculation mode

Monthly Savings Plan

In this mode, the program calculates the required monthly increase in savings.

The algorithm accepts initial saving amount (a), target savings (S_n), and number of months (n). Then it handles special cases such as $n = 1$ and constant savings ($d = 0$). Afterwards we use the derived arithmetic series formula to compute the required difference. And a month-by-month savings table is generated using a finite loop demonstrating finite arithmetic progression.

Month Calculation Mode

In this mode, the program determines the number of months required to reach a target amount when initial saving amount and difference are known. The algorithm initializes total savings to zero then adds each month's saving discretely to the total. The loop continues until the target is reached or a predefined upper limit is exceeded.

This method demonstrates the summation of a discrete sequence.

Sample Output / Tables

Monthly Savings Plan:

===WELCOME TO SAVINGS PLANNER===

Select an option:

For Monthly Savings Plan, Enter 1

For Month calculation, Enter 2

To Exit Savings Planner, Enter 3

Enter your choice: 1

Enter the current amount you are saving on a monthly basis (PKR): 1000

Enter your target savings amount (PKR): 120000

Enter the number of months you want to achieve this target in: 12

--- Savings Recommendation ---

Recommended monthly increase: PKR 1637.00

Your Monthly Savings Plan

Month	Monthly Saving (PKR)	Total Savings (PKR)
1	1000.00	1000.00
2	2637.00	3637.00
3	4274.00	7911.00
4	5911.00	13822.00
5	7548.00	21370.00
6	9185.00	30555.00
7	10822.00	41377.00
8	12459.00	53836.00
9	14096.00	67932.00
10	15733.00	83665.00
11	17370.00	101035.00
12	19007.00	120042.00

You must increase your savings by PKR 1637.00 every month to reach the target.

Select an option:

For Monthly Savings Plan, Enter 1

For Month calculation, Enter 2

To Exit Savings Planner, Enter 3

Month Prediction:

Enter your choice: 2

Enter your current monthly savings (PKR): 200

Enter monthly increase in saving (PKR): 150

Enter target amount (PKR): 25000

Savings Predictor table

Month	Monthly Saving (PKR)	Total Savings (PKR)
1	200.00	200.00
2	350.00	550.00
3	500.00	1050.00
4	650.00	1700.00
5	800.00	2500.00
6	950.00	3450.00
7	1100.00	4550.00
8	1250.00	5800.00
9	1400.00	7200.00
10	1550.00	8750.00
11	1700.00	10450.00
12	1850.00	12300.00
13	2000.00	14300.00
14	2150.00	16450.00
15	2300.00	18750.00
16	2450.00	21200.00
17	2600.00	23800.00
18	2750.00	26550.00

You will reach your target in 18 months.

Your Total savings at that time: PKR 26550.00

Select an option:

For Monthly Savings Plan, Enter 1

For Month calculation, Enter 2

To Exit Savings Planner, Enter 3

Enter your choice: 3

Thank you For Choosing Savings Planner!

===Goodbye!===

Conclusion

This project demonstrates the effective application of Discrete Mathematical Structures in real-world financial planning. By modelling monthly savings as an arithmetic progression and applying finite series formulas, savings targets can be systematically analyzed and achieved.

The Smart Savings Planner highlights the importance of sequences, series, and discrete summation in solving practical problems, reinforcing the relevance of discrete mathematics in computational and everyday contexts.

Java Source Code

```
import java.util.Scanner;

public class SavingPlanner {

    private static final int savingsPlan = 1;

    private static final int monthCalculation = 2;

    private static final int exitProgram = 3;

    public static void printMenu() {

        System.out.println("Select an option:");

        System.out.println("For Monthly Savings Plan, Enter 1");

        System.out.println("For Month calculation, Enter 2");

        System.out.println("To Exit Savings Planner, Enter 3\n");

    }

    public static void main(String[] args) {

        Scanner h = new Scanner(System.in);

        int choice;

        System.out.println("===WELCOME TO SAVINGS PLANNER===");

        String msg = "Enter your choice: ";
```

```

do {

    printMenu();

    choice = getMenuChoice(h);

    switch (choice) {

        case savingsPlan:

            savingsPlanner(h);

            break;

        case monthCalculation:

            monthCalculator(h);

            break;

        case exitProgram:

            System.out.println("Thank you For Choosing Savings Planner!");

            break;

        default:

            System.out.println("Invalid Choice Entered");

    }

} while (choice != exitProgram);

System.out.println("===Goodbye!===");

h.close();

}

public static void savingsPlanner(Scanner h) {

    double initialAmount = getPositiveDouble(h, "Enter the current amount you are saving on a monthly basis (PKR): ");

    double targetAmount = getPositiveDouble(h, "Enter your target savings amount (PKR): ");

    int months = getPositiveInt("Enter the number of months you want to achieve this target in: ", h);

    if (months == 1) {

        if (initialAmount >= targetAmount) {

            System.out.println("Your target is already achieved in 1st Month!");

        } else {

            System.out.println("Your target cannot be achieved in 1 month!");

        }

    }

    return;

}

```



```

        if (months * initialAmount >= targetAmount) {
            System.out.println("You will reach your target without increasing your monthly savings");
            APTable(initialAmount, months, 0);
            return;
        }
        double difference = ((2 * targetAmount) / months - 2 * initialAmount) / (months - 1);
        difference = Math.ceil(difference);
        System.out.println("\n--- Savings Recommendation ---");
        System.out.printf("Recommended monthly increase: PKR %.2f\n", difference);
        APTable(initialAmount, months, difference);
    }

    public static void APTable(double a, int n, double d) {
        System.out.println("Your Monthly Savings Plan");
        double sum = 0.0;
        double term = a;
        System.out.printf("%-10s %-20s %-20s\n", "Month", "Monthly Saving (PKR)", "Total Savings (PKR)");
        for (int i = 1; i <= n; i++) {
            sum += term;
            System.out.printf("%-10d %-20.2f %-20.2f\n", i, term, sum);
            term += d;
        }
        if (d > 0) {
            System.out.printf("\nYou must increase your savings by PKR %.2f every month to reach the target.\n\n",
d);
        }
    }

    public static void monthCalculator(Scanner h) {
        double a = getPositiveDouble(h, "Enter your current monthly savings (PKR): ");
        double d = getPositiveDouble(h, "Enter monthly increase in saving (PKR): ");
        double target = getPositiveDouble(h, "Enter target amount (PKR): ");
        double sum = 0.0;
        double savings = a;
        int month = 0;
    }

```

```

if (d == 0 && a < target) {
    System.out.println("Target cannot be achieved without increasing monthly savings.");
    return;
}

System.out.println("Savings Predictor table");

System.out.printf("%-10s %-20s %-20s\n", "Month", "Monthly Saving (PKR)", "Total Savings (PKR)");

while (sum < target && month <= 600) {
    month++;
    sum += savings;
    System.out.printf("%-10d %-20.2f %-20.2f\n", month, savings, sum);
    savings += d;
}

if (sum >= target) {
    System.out.printf("You will reach your target in %d months.\n", month);
    System.out.printf("Your Total savings at that time: PKR %.2f\n", sum);
} else {
    System.out.println("Target not achievable within a reasonable time frame.");
}
}

public static int getMenuChoice(Scanner h) {
    while (true) {
        System.out.print("Enter your choice: ");
        if (h.hasNextInt()) {
            int choice = h.nextInt();
            if (choice >= 1 && choice <= 3) return choice;
            System.out.println("Please enter 1, 2, or 3 only.");
        } else {
            System.out.println("Invalid input.");
            h.next();
        }
    }
}

public static int getPositiveInt(String msg, Scanner h) {

```

```
while (true) {  
    System.out.print(msg);  
    if (h.hasNextInt()) {  
        int value = h.nextInt();  
        if (value > 0) return value;  
        System.out.println("Enter a positive integer.");  
    } else {  
        System.out.println("Invalid input.");  
        h.next();  
    }  
}  
  
public static double getPositiveDouble(Scanner h, String msg) {  
    while (true) {  
        System.out.print(msg);  
        if (h.hasNextDouble()) {  
            double value = h.nextDouble();  
            if (value > 0) return value;  
            System.out.println("Enter a positive Value");  
        } else {  
            System.out.println("Try again! ");  
            h.next();  
        }  
    }  
}
```